Sprint Progress Report 4

Sprint Overview

During Sprint 4, our team focused on solidifying the foundational architecture of our backend and refining the core user interfaces on the frontend. We successfully implemented a clean, multi-layered backend architecture (Data Validation, ORM, Data Access Layer, and Business Logic) and secured it with comprehensive unit testing. On the frontend, we polished the essential user journeys to prepare for the upcoming API integrations.

1. Class Presentation

Our team completed the in-class presentation requirement

2. Work Completed

Frontend Development:

- **Page Completion & UI/UX Refinement**: Successfully completed and visually improved four core screens: the Landing Home page, the secure Login page, the authenticated user Dashboard, and the Add Expense form interface.
- Focused on responsive design and component modularity to ensure these pages are fully prepared to consume backend REST APIs in the next sprint.

Backend Architecture & Core Logic:

- **Data Validation Layer (Schemas):** Implemented Pydantic models (DTOs) to enforce strict incoming data validation for users, groups, expenses, and settlements, ensuring that the backend only processes clean, structured JSON payloads.
- **Database Models (ORM):** Finalized the SQLAlchemy ORM definitions, correctly mapping our relational database schema (including critical foreign key constraints and cascading behaviors).
- **Data Access Layer (Repositories)**: Abstracted all direct database interactions into dedicated Repository classes (UserRepository, ExpenseRepository, etc.). We successfully implemented atomic database transactions (using flush, commit, and rollback) to guarantee ledger consistency when saving main expenses alongside multiple split participants.
- **Business Logic Layer (Services):** Completed the majority of the Service layer, which now acts as the central "brain" of the application, handling complex operations without being tightly coupled to raw SQL queries.

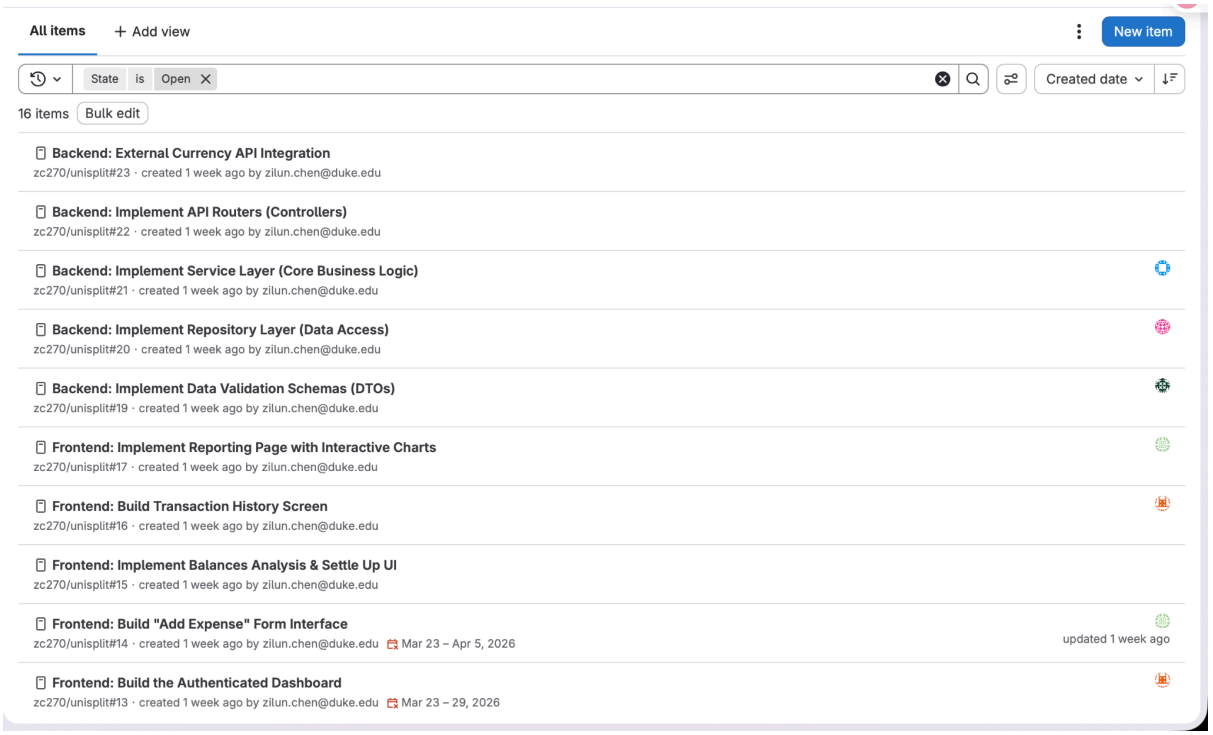
Testing & Quality Assurance:

- Developed comprehensive unit tests for the models, schemas and repositories.
- Utilized an in-memory SQLite database (sqlite:///:memory:) combined with pytest fixtures to test transactional safety, cascading deletes, and foreign key constraints in a completely isolated environment, ensuring our financial ledger operations are 100% reliable.

3. GitLab Issues & Work Division

Tasks for Sprint 4 were tracked and assigned via GitLab issues:

- **Frontend UI/UX:** Refine Dashboard, Login, Home, and Add Expense pages.
- **Backend Foundation:** Implement Pydantic Schemas, SQLAlchemy Models, and Repository layer with transaction safety.
- **Backend Logic & Testing:** Develop the Service layer logic and write `pytest` suites for schemas and repositories



4. Project Artifacts Updates

Our project artifacts were updated to reflect the current progress of the sprint:

- **Codebase Updates (Backend):** Restructured the backend directory to follow a strict layered architecture (/app/models, /app/schemas, /app/repositories, /app/services, /app/tests).
- **Codebase Updates (Frontend):** Committed the polished versions of the core React/Vue component pages.
- **Testing Artifacts:** Added automated test scripts that validate data integrity and transactional rollbacks.

5. Weekly Meeting Summary

Held weekly meeting on Apr 5th with TA. We demonstrated the newly refined frontend interfaces and discussed our backend architectural shift toward the Repository Pattern. We also outlined our upcoming plans for role-based access control (Admin vs. Regular User) and third-party API integration.

6. Challenges & Learnings

- **Challenge - Transaction Management:** Ensuring that a master expense record and its multiple split details were saved simultaneously without causing partial database writes in the event of an error.
- **Learning - The Repository Pattern:** We successfully overcame this challenge by implementing the Repository pattern and mastering SQLAlchemy's session management.
- **Challenge - Type Checking:** Reconciling strict Pydantic types with SQLAlchemy ORM string requirements.
- **Learning:** Learned how to properly cast and map data transfer objects (DTOs) to database entities before persistence.

7. Next Steps (Sprint 5 Plan)

Moving into Sprint 5, we will focus on feature expansion and third-party integrations:

- **Role-Based Access Control (RBAC):** Implement distinct views and privileges for Administrative Users versus Regular Users, which will primarily be reflected in differentiated Login routing and specialized Dashboard analytics.
- **Transaction History:** Develop and integrate the "View Transactions" feature, allowing users to scroll through a detailed ledger of past group expenses and settlements.
- **Membership Mechanism:** Introduce a premium/membership tier system to the application architecture to unlock advanced features for subscribed users.
- **External API Integration:** Integrate a live Currency Exchange Rate REST API. We plan to display this real-time data within a convenient UI modal/popup, assisting users in accurately calculating international or travel expenses.